UNIT-4
TUPLES, SETS and DICTIONARIES

Tuples:

- Tuple is a sequence of elements. A tuple is immutable, unlike a list in Python. By immutable, we mean that it is not possible to update a tuple after declaring one. The elements inside a tuple can be any Python object such a string, integer etc. Tuples are surrounded by parenthesis.
- **Examples:**

```
>>> t1 = (1, "abc", 3.0)
>>> t1
(1, 'abc', 3.0)
>>> type(t1)
<type 'tuple'>
```

Declaring a Tuple:

- Empty Tuple is declared by empty parenthesis.

```
>>> t2 = ()
>>> t2
()
>>> type(t2)
<type 'tuple'>
```

- It is important to note that declaring a tuple consisting a single element requires a comma after the first element.

```
>>> t3 = (1,)
>>> t3
(1,)
>>> type(t3)
<type 'tuple'>
```

- If we miss out on that comma, **t3** would become an integer because 1 itself is an integer.

```
>>> t3 = (1)
>>> type(t3)
<type 'int'>
>>> t3 = ('abc')
>>> type(t3)
<type 'str'>
```

Accessing elements of a Tuple:

- We use square brackets along with the index which we want to access.
- **Examples:**

```
>>> t1 = (1, "abc", 3.0)
>>> t1[0]
1
>>> t1[1]
'abc'
>>> t1[2]
3.0
```

```
>>> t1[3]
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
IndexError: tuple index out of range
```

- If you try accessing some index that doesn't exist, Python raises appropriate error. We can also access elements using negative indexing.

```
>>> t1[-1]
3.0
>>> t1[-2]
'abc'
```

- Index table indicating positive and negative index values.

+ ve Index	0	1	2
Tuple=	(1,	"abc",	3.0)
- ve Index	-3	-2	-1

Updating elements of a Tuple:

- We saw in the definition of a tuple that tuples are immutable. It is not possible to update them once declared. but what happens if we try to do so?

```
>>> t1 = (1, "abc", 3.0)
>>> t1[1] = 'xyz'
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

- If we try to update the element by using index, it raises an error saying that 'tuple' object does not support item assignment.
- However, if you add mutable elements such as a list inside a tuple, we can mutate them. For example:

```
>>> a = [1, 2]
>>> b = [4, 5]
>>> t = (a, b)
>>> t
([1, 2], [4, 5])
>>> a.append(3)
>>> t
([1, 2, 3], [4, 5])
```

- A tuple can also be nested, i.e. just like a list inside a tuple, we can have tuples inside a tuple.

```
>>> a = (1, 2)
>>> b = (4, 5)
>>> t = (a, b)
>>> t
((1, 2), (4, 5))
```

- However, now that tuples are not mutable, we cannot update the tuple inside a tuple either.

- If we really want to update a tuple, we can use that tuple to create another tuple.

```
>>> t2 = (t[0], (3), t[1])
>>> t2
((1, 2), 3, (4, 5))
```

Deleting a Tuple:

- Since tuples are immutable, it is not possible to delete individual elements of a tuple, however, we can delete the whole tuple itself.

```
>>> t = (1, "abc", 3.0)
>>> del t
>>> t  #If we try to access the tuple after deleting it, python will raise an error
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 't' is not defined
```

Slicing a Tuple:

- Tuple slicing is similar to string slicing. Using slicing we can extract out elements of any tuple. We have to provide the starting index and the ending index i.e. **tuple[start:end]**

```
>>> t = ('s', 'u', 's', 'a', 'n')
>>> t[1:4]
('u', 's', 'a')
```

- It starts from **start** and ends right before **end**. It won't print the element at the index **end**. As in the example above, we have **n** at the 4th index of tuple **t**, however, it printed only till **a**.
- If the **start** is out of the range of the tuple, then it prints empty tuple.

```
>>> t[5:7]
()
```

- If you don't provide **start**, then it will print from **start** till **end-1** element.

```
>>>  t[:3]
('s', 'u', 's')
```

- Similarly, if **end** is not provided, it prints till the last element of the tuple.

```
>>>  t[2:]
('s', 'a', 'n')
```

- If there is no **start** and **end** It prints the whole tuple as is.

```
>>> t[:]
('s', 'u', 's', 'a', 'n')
```

Operations on tuple:

- **Addition:**
  Using the addition operator, with two or more tuples, adds up all the elements into a new tuple. For example,

```
>>> t = (2, 5, 0) + (1, 3) + (4,)
>>> print t
(2, 5, 0, 1, 3, 4)
```

- **Multiplication**
  Multiplying a tuple by any **integer, x** will simply create another tuple with all the elements from the first tuple being repeated **x** number of times. For example, t*3 means, elements of tuple t will be repeated 3 times.

```
>>> t = (2, 5)
>>> print t*3
(2, 5, 2, 5, 2, 5)
```

- **Member ship operation(in keyword):**
  in keyword, can not only be used with tuples, but also with strings and lists too. It is used to check, if any element is present in the sequence or not. It returns True if the element is found, otherwise False. For example,

```
>>> t = (1, 2, 3, 6, 7, 8)
>>> 2 in t
True
>>> 5 in t
False
```

- **Iterating Through a Tuple:**
  Using a for loop we can iterate though each item in a tuple.

```
for name in ('John','Kate'):
     print("Hello",name)
```

Output:

```
Hello John
Hello Kate
```

Functions of Tuple:

- **len() function**
  As you might have already guessed, this function is used to get the number of elements inside any tuple.

```
>>> t = 1, 2, 3
>>> print len(t)
 3
```

- **cmp() function**
  This is used to compare two tuples. It will return either 1, 0 or -1, depending upon whether the two tuples being compared are similar or not.

The cmp() function takes two tuples as arguments, where both of them are compared. If **T1** is the first tuple and **T2** is the second tuple, then:

- if **T1 > T2**, then cmp(T1, T2) returns **1**
- if **T1 = T2**, then cmp(T1, T2) returns **0**
- if **T1 > T2**, then cmp(T1, T2) returns **-1**

- **max() and min() function**
  To find the maximum value in a tuple, we can use the max() function, while for finding the minimum value, min() function can be used.

```
>>> t = (1, 4, 2, 7, 3, 9)
>>> print max(t)
9
>>> print min(t)
1
```

Methods of tuple:

- Methods that add items or remove items are not available with tuple. Only the following two methods are available.

Method	Description
count(x)	Return the number of items that is equal to x
index(x)	Return index of first item that is equal to x

- Some examples of Python tuple methods:

```
 my_tuple = ('a','p','p','l','e',)

# Count
# Output: 2
print(my_tuple.count('p'))

# Index
# Output: 3
print(my_tuple.index('l'))
```

Advantages of Tuple over List

Since, tuples are quite similar to lists, both of them are used in similar situations as well. However, there are certain advantages of implementing a tuple over a list. Below listed are some of the main advantages:

- We generally use tuple for heterogeneous (different) datatypes and list for homogeneous (similar) datatypes.
- Since tuple are immutable, iterating through tuple is faster than with list. So there is a slight performance boost.
- Tuples that contain immutable elements can be used as key for a dictionary. With list, this is not possible.
- If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.

Tuple vs List:

- The key difference is that tuples are **immutable** . This means that you cannot change the values in a tuple once you have created it. As a list is **mutable** , it can't be used as a key in a dictionary, whereas a tuple can be used.
- The literal **syntax** of tuples is shown by parentheses **( )** whereas the literal syntax of lists is shown by square brackets **[ ]** .
- Tuples are **heterogeneous** data structures (i.e., their entries have different meanings), while lists are **homogeneous** sequences.
- Lists are for **variable length** , tuples are for **fixed length** .
- Tuples show **structure** whereas lists show **order**..

****

Sets:

- A set is an unordered collection of items. Every element is unique (no duplicates) and must be immutable (which cannot be changed).
- However, the set itself is mutable. We can add or remove items from it.
- Sets can be used to perform mathematical set operations like union, intersection, symmetric difference etc.

Creating a set:

- A set is created by placing all the items (elements) inside curly braces {}, separated by comma or by using the built-in function set().
- It can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have a mutable element, like list, set or dictionary, as its element.

  set of integers
  ```
  my_set = {1, 2, 3}
  ```
  set of mixed datatypes
  ```
  my_set = {1.0, "Hello", (1, 2, 3)}
  ```
  set do not have duplicates
  ```
  my_set = {1,2,3,4,3,2}
  print(my_set) #{1, 2, 3, 4}
  ```

set cannot have mutable items, here [3, 4] is a mutable list

```
my_set = {1, 2, [3, 4]}
```

we can make set from a list

```
my_set = set([1,2,3,2])
print(my_set) #{1, 2, 3}
```

Creating an empty set:

- Empty curly braces {} will make an empty dictionary in Python. To make a set without any elements we use the set() function without any argument.

```
# initialize a with {}
a = {}

# check data type of a
# Output: <class 'dict'>
print(type(a))

# initialize a with set()
a = set()

# check data type of a
# Output: <class 'set'>
print(type(a))
```

Changing a set:

- Sets are mutable. But since they are unordered, indexing have no meaning.
- We cannot access or change an element of set using indexing or slicing. Set does not support it.
- We can add single element using the add() method and multiple elements using the update() method. The update() method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

```
# initialize my_set
my_set = {1,3}
print(my_set)

# add an element
# Output: {1, 2, 3}
my_set.add(2)
print(my_set)

# add multiple elements
# Output: {1, 2, 3, 4}
my_set.update([2,3,4])
print(my_set)

# add list and set
# Output: {1, 2, 3, 4, 5, 6, 8}
my_set.update([4,5], {1,6,8})
print(my_set)
```

removing elements from a set:

- A particular item can be removed from set using methods, discard() and remove().
- The only difference between the two is that, while using discard() if the item does not exist in the set, it remains unchanged. But remove() will raise an error in such condition.
- The following example will illustrate this.

```
my_set = {1, 3, 4, 5, 6}
print(my_set)

# discard an element
# Output: {1, 3, 5, 6}
my_set.discard(4)
print(my_set)

# remove an element
# Output: {1, 3, 5}
my_set.remove(6)
print(my_set)

# discard an element not present in my_set
# Output: {1, 3, 5}
my_set.discard(2)
print(my_set)
```

- Similarly, we can remove and return an item using the pop() method.
- Set being unordered, there is no way of determining which item will be popped. It is completely arbitrary.
- We can also remove all items from a set using clear().

```
# initialize my_set
# Output: set of unique elements
my_set = set("HelloWorld")
print(my_set)

# pop an element
# Output: random element
print(my_set.pop())

# pop another element
# Output: random element
my_set.pop()
print(my_set)

# clear my_set
#Output: set()
my_set.clear()
print(my_set)
```

Python Set Operations:

Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods.

Operation	Symbol	Method	Description
Union	|	union()	Union of A and B is a set of all elements from both sets.
Intersection	&	intersection()	Intersection of A and B is a set of elements that are common in both sets.
Difference	-	difference()	Difference of A and B (A - B) is a set of elements that are only in A but not in B. Similarly, B - A is a set of element in B but not in A.
Symmetric Difference	^	symmetric_difference()	Symmetric Difference of A and B is a set of elements in both A and B except those that are common in both.

Examples:

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

Union:

```
print(A|B) # {1, 2, 3, 4, 5, 6, 7, 8}
print(B|A) # {1, 2, 3, 4, 5, 6, 7, 8}
A.union(B) # {1, 2, 3, 4, 5, 6, 7, 8}
B.union(A) # {1, 2, 3, 4, 5, 6, 7, 8}
```

Intersection:

```
print(A&B) #{4, 5}
print(B&) #{4, 5}
A.intersection(B) #{4, 5}
B.intersection(A) #{4, 5}
```

Difference:

```
print(A-B) # {1, 2, 3}
print(B-A) # {8, 6, 7}
A.difference(B) # {1, 2, 3}
B.difference(A) # {8, 6, 7}
```

Symmetric Difference:

```
print(A^B) # {1, 2, 3, 6, 7, 8}
print(B^A) # {1, 2, 3, 6, 7, 8}
A.symmetric_difference(B) # {1, 2, 3, 6, 7, 8}
B.symmetric_difference(A) # {1, 2, 3, 6, 7, 8}
```

Methods of set:

Method	Description	Syntax	Example
add()	Adds an element to the set	set.add(elem)	vowels = {'a', 'e', 'i', 'u'} vowels.add('o') print(vowels) **Output:** {'a', 'e', 'i', 'o','u'}
clear()	Removes all elements from the set	set.clear()	vowels.clear() print(vowels) **Output:** set()
copy()	Returns a copy of the set	set.copy()	num = {1, 2, 3, 4} new_num = num.copy() print(new_num) **Output:** {1, 2, 3, 4}
difference()	Returns the difference of two or more sets as a new set	A.difference(B)	A = {'a', 'b', 'c', 'd'} B = {'c', 'f', 'g'} print(A.difference(B)) **Output:** {'b', 'a', 'd'}
difference_update()	Removes all elements of another set from this set	A.difference_update(B)	A = {'a', 'c', 'g', 'd'} B = {'c', 'f', 'g'} result = A.difference_update(B) print('A = ', A) print('B = ', B) print('result = ', result) **Output:** A = {'d', 'a'} B = {'c', 'g', 'f'} result = None
discard()	Removes an element from the set if it is a member. (Do nothing if the element is not in set)	s.discard(element)	numbers = {2, 3, 4, 5} numbers.discard(3) print('numbers = ', numbers) **Output:** numbers = {2, 4, 5}
intersection()	Returns the intersection of two sets as a new set	A.intersection(B)	A = {2, 3, 5, 4} B = {2, 5, 100} print(B.intersection(A)) **Output:** {2, 5}

intersection_update()	Updates the set with the intersection of itself and another	A.intersection_update(B)	A = {1, 2, 3, 4} B = {2, 3, 4, 5} result = A.intersection_update(B) print('result =', result) print('A =', A) print('B =', B) **Output:** result = None A = {2, 3, 4} B = {2, 3, 4, 5}
isdisjoint()	Returns True if two sets have a null intersection	A.isdisjoint(B)	A = {1, 2, 3, 4} B = {5, 6, 7} print(A.isdisjoint(B)) **Output:** True
issubset()	Returns True if another set contains this set	A.issubset(B)	A = {1, 2, 3} B = {1, 2, 3, 4, 5} print(A.issubset(B)) **Output:** True
issuperset()	Returns True if this set contains another set	A.issuperset(B)	A = {1, 2, 3} B = {1, 2, 3, 4, 5} print(B.issuperset(A)) **Output:** True
pop()	Removes and returns an arbitary set element. Raise KeyError if the set is empty	set.pop()	A ={'a', 'b', 'c', 'd'} print(A.pop()) **Output:** c
remove()	Removes an element from the set. If the element is not a member, raise a KeyError	set.remove(element)	vowels = {'a', 'e', 'i', 'u'} vowels.remove('e') print(vowels) **Output:** {'a', 'i', 'u'}
symmetric_difference()	Returns the symmetric difference of two sets as a new set	A.symmetric_ difference(B)	A = {'a', 'b', 'c', 'd'} B = {'c', 'd', 'e' } print(A.symmetric_difference(B)) **Output:** {'a', 'e', 'b'}

symmetric_ difference_update()	Updates a set with the symmetric difference of itself and another	A.symmetric_ difference_update(B)	A = {'a', 'c', 'd'} B = {'c', 'd', 'e' } r= A.symmetric_difference_ update(B) print('A = ', A) print('B = ', B) print('result = ', result) **Output:** A = {'a', 'e'} B = {'e', 'c', 'd'} result = None
union()	Returns the union of sets in a new set	A.union(B)	A = {'a', 'c', 'd'} B = {'c', 'd', 2 } print(A.union(B)) **Output:** {'a', 2, 'c', 'd'}
update()	Updates the set with the union of itself and others	A.update(B)	A = {'a', 'b'} B = {1, 2, 3} result = A.update(B) print('A =',A) print('B =',B) print('result =',result) **Output:** A = {'a', 2, 3, 1, 'b'} B = {1, 2, 3} result = None

Functions of sets:

Function	Description	Syntax	Example
all()	Return True if all elements of the set are true (or if the set is empty).	all(iterable)	s1= [1, 3, 4, 5] print(all(s1))#**True** s2= [1, 3, 4, 0] print(all(s2))#**False**
any()	Return True if any element of the set is true. If the set is empty, return False.	any(iterable)	s1 = [1, 3, 4, 0] print(any(s1))#**True** s2 = [0, False] print(any(s2))#**False**
enumerate()	Return an enumerate object. It contains the index and value of all the items of set as a pair.	enumerate(iterable, start=0)	vowels = ['a', 'e', 'i'] enumVowels = enumerate(vowels) print(type(enumVowels)) <class 'enumerate'> print(list(enumVowels)) [(0, 'a'), (1, 'e'), (2, 'i')]

len()	Return the length (the number of items) in the set.	len(iterable)	s1={1,2,3,4} s2={1,1,1,2,3} print(len(s1))#**4** print(len(s2))#**3**
max()	Return the largest item in the set.	max(iterable)	s1={1,2,3,4} print(max(s1))#**4**
min()	Return the smallest item in the set.	min(iterable)	s1={1,2,3,4} print(min(s1))#**1**
sorted()	Return a new sorted list from elements in the set(does not sort the set itself).	sorted(iterable, reverse=flag)	s1 = {'e', 'a', 'u', 'o', 'i'} print(sorted(s1))# ['a', 'e', 'i', 'o', 'u'] print(sorted(s1, reverse=True)) #['u', 'o', 'i', 'e', 'a']
sum()	Retrun the sum of all elements in the set.	sum(iterable)	num={1,2,3,4,5} print(sum(num))#**15**

****

Dictionaries:

- Python dictionary is an unordered collection of items. While other compound data types have only value as an element, a dictionary has a key: value pair.
- Dictionaries are optimized to retrieve values when the key is known.

Creating Dictionary:

- Creating a dictionary is as simple as placing items inside curly braces {} separated by comma.
- An item has a key and the corresponding value expressed as a pair, key: value.
- While values can be of any data type and can repeat, keys must be of immutable type (string, number or tuple with immutable elements) and must be unique.

```
# empty dictionary
my_dict = {}
# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}
# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}
# using dict()
my_dict = dict({1:'apple', 2:'ball'})
# from sequence having each item as a pair
my_dict = dict([(1,'apple'), (2,'ball')])
```

- we can also create a dictionary using the built-in function dict().

Accessing elements from Dictionary:

- While indexing is used with other container types to access values, dictionary uses keys. Key can be used either inside square brackets or with the get() method.
- The difference while using get() is that it returns None instead of KeyError, if the key is not found.

```
my_dict = {'name':'Jack', 'age': 26}
print(my_dict['name'])
print(my_dict.get('age'))
```

output:

```
Jack
26
```

Changing Elements of Dictionary:

- Dictionary are mutable. We can add new items or change the value of existing items using assignment operator.
- If the key is already present, value gets updated, else a new key: value pair is added to the dictionary.

```
my_dict = {'name':'Jack', 'age': 26}
```

update value

```
my_dict['age'] = 27
print(my_dict)
#Output: {'age': 27, 'name': 'Jack'}
```

add item

```
my_dict['address'] = 'Downtown'
print(my_dict)
#Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
```

- We can remove a particular item in a dictionary by using the method pop(). This method removes as item with the provided key and returns the value.
- The method, popitem() can be used to remove and return an arbitrary item (key, value) form the dictionary. All the items can be removed at once using the clear() method.
- We can also use the del keyword to remove individual items or the entire dictionary itself.

create a dictionary

```
squares = {1:1, 2:4, 3:9, 4:16, 5:25}
```

remove a particular item

```
print(squares.pop(4))
# Output: 16
print(squares)
# Output: {1: 1, 2: 4, 3: 9, 5: 25}
```

remove an arbitrary item

```
print(squares.popitem())
print(squares)
# Output: (1, 1)
# Output: {2: 4, 3: 9, 5: 25}
```

delete a particular item

```
del squares[5]
print(squares)
# Output: {2: 4, 3: 9}
```

remove all items

```
squares.clear()
print(squares)
# Output: {}
```

delete the dictionary itself

```
del squares
print(squares)
# Throws Error
```

Methods of Dictionary:

Method	Description	Syntax	Example
clear()	Remove all items form the dictionary.	dict.clear()	d = {1: "one", 2: "two"} d.clear() print('d =', d) **Output:** d = {}
copy()	Return a shallow copy of the dictionary.	dict.copy()	original = {1:'one', 2:'two'} new = original.copy() print('Orignal: ', original) print('New: ', new) **Output:** Orignal: {1: 'one', 2: 'two'} New: {1: 'one', 2: 'two'}
fromkeys()	Return a new dictionary with keys from seq and value equal to v (defaults to None).	dict.fromkeys (sequence[,value])	keys = {'a', 'e', 'i', 'o', 'u' } vowels = dict.fromkeys(keys) print(vowels) **Output:** {'u': None, 'o': None, 'i': None, 'a': None, 'e':None}
get()	Return the value of key. If key does not exit, return d (defaults to None).	dict.get(key[,value])	person = {'name': 'Phill', 'age': 22} print('Name: ', person.get('name')) print('Age: ', person.get('age')) **Output:** Name: Phill Age: 22

items()	Return a new view of the dictionary's items (key, value).	dict.items()	sales={'apple': 2,'orange': 3,'grapes': 4} print(sales.items()) **Output:** dict_items([('apple',2),('orange',3),('grapes',4)])
keys()	Return a new view of the dictionary's keys.	dict.keys()	p= {'name': 'Phill', 'age': 22, 'salary': 3500.0} print(p.keys()) **Output:** dict_keys(['age', 'name', 'salary'])
pop()	Remove the item with key and return its value or d if key is not found. If d is not provided and key is not found, raises KeyError.	dict.pop(key[,default])	sales = { 'apple': 2, 'orange': 3, 'grapes': 4 } element = sales.pop('apple') print('The popped element is:', element) print('The dictionary is:', sales) **Output:** The popped element is: 2 The dictionary is: {'grapes': 4, 'orange': 3}
popitem()	Remove and return an arbitary item (key, value). Raises KeyError if the dictionary is empty.	dict.popitem()	person= {'name': 'Phill', 'age': 22, 'salary': 3500.0} result = person.popitem() print('person = ',person) print('Return Value = ',result) **Output:** person = {'salary': 3500.0, 'age': 22} Return Value = ('name', 'Phill')
setdefault()	If key is in the dictionary, return its value. If not, insert key with a value of d and return d (defaults to None).	dict.setdefault (key[, default_value])	person = {'name': 'Phill', 'age': 22} age = person.setdefault('age') print('person = ',person) print('Age = ',age) **Output:** person = {'name': 'Phill', 'age': 22} Age = 22

update()	Update the dictionary with the key/value pairs from other, overwriting existing keys.	dict.update([other])	d = {1: "one", 2: "three"}
			d1 = {2: "two"}
			# updates the value of key 2
			d.update(d1)
			print(d)
			Output:
			{1: 'one', 2: 'two'}

values()	Return a new view of the dictionary's values	dict.values()	sales = { 'apple': 2, 'orange': 3, 'grapes': 4 } print(sales.values()) **Output:** dict_values([2, 4, 3])

Functions of Dictionary:

Function	Description	Syntax	Example
all()	Return True if all keys of the dictionary are true (or if the dictionary is empty).	all(iterable)	s = {0: 'False', 1: 'False'} print(all(s))#**False** s = {1: 'True', 2: 'True'} print(all(s))#**True**
any()	Return True if any key of the dictionary is true. If the dictionary is empty, return False.	any(iterable)	d = {0: 'False'} print(any(d))#**False** d = {0: 'False', 1: 'True'} print(any(d))#**True**
len()	Return the length (the number of items) in the dictionary.	len(iterable)	d = {1: 'one', 2: 'two'} print(d, 'length is', len(d)) # **{1: 'one', 2: 'two'} length is 2**
sorted()	Return a new sorted list of keys in the dictionary.	sorted(iterable[, key][, reverse])	d= {'e': 1, 'a': 2, 'u': 3, 'o': 4, 'i': 5} print(sorted(d, reverse=True)) # **['u', 'o', 'i', 'e', 'a']**

Other Dictionary Operations

Dictionary Membership Test:

We can test if a key is in a dictionary or not using the keyword in. Notice that membership test is for keys only, not for values.

- **Example:**
  squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
  print(1 in squares)
  print(2 not in squares)
  print(49 in squares)
  Output:
  True
  True
  False

Iterating Through a Dictionary:

Using a for loop we can iterate though each key in a dictionary.

Example:

```
squares = {1: 1, 3: 9, 5: 25, 7: 49}
for i in squares:
    print(squares[i])
```

Output:

1
9
81
25
49

Dictionary Comprehension:

- Dictionary comprehension is an elegant and concise way to create new dictionary from an iterable in Python.
- Dictionary comprehension consists of an expression pair (key: value) followed by for statement inside curly braces {}.
- Here is an example to make a dictionary with each item being a pair of a number and its square.
- **Example:**
  ```
  squares = {x: x*x for x in range(6)}
  print(squares)
  ```
  Output:
  {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

- The above code is equivalent to
  ```
  squares = {}
  for x in range(6):
        squares[x] = x*x
  ```

****